恒生电子股份有限公司

编码规范 JAVA 分则

2007 年 9 月

目 录

目的.....1
1 命名规范.....1
1.1 包.....1
1.2 变量.....1
1.3 常量.....2
1.4 类和接口.....2
1.5 方法.....3
1.6 控件.....3
2 编码规范.....6
2.1 变量的声明.....6
2.2 类及接口的声明.....7
2.3 复合语句.....8
2.4 if, if-else-if else 语句.....8
2.5 for 语句.....9
2.6 while 语句.....10
2.7 do-while 语句.....10
2.8 switch 语句.....10
2.9 try-catch 语句.....11
3 源文件编写规范.....12
3.1 源文件命名.....12
3.2 源文件的结构.....12
4 注释和JavaDoc.....13
4.1 包注释.....13
4.2 类注释.....13
4.3 方法注释.....14
4.4 行尾注释.....14
4.5 嵌入式注释.....15
5 排版规范.....15
5.1 大小写.....15
5.2 代码缩进、空行及空格的使用.....16
5.2.1 代码的缩进.....16
5.2.2 行长度.....16
5.2.3 换行.....16
5.2.4 空行.....18
5.2.5 空格.....19
附录 Web/Struts编码规则.....20
6.1 Web目录命名规则.....20
6.2 功能点link命名规则.....21
6.3 View/JSP命名规则.....21
6.4 Action命名规则.....22
6.5 包命名规则.....23
6.6 Form命名规则.....23

目的

本文旨在为Java编程提供一系列的编码规范。这一规范并非针对编程的逻辑性，而是针对它的物理结构和外观，使得代码更容易被阅读、理解及维护。

1 命名规范

1.1 包

一个单独的包命名的前缀通常全部由小写的ASCII字母组成，且应为最高层域名之一，通常是com、edu、gov、mil、net、org或是1981年的ISO 标准3166中详细说明的识别国家的英文双字母编码之一。

包名称的后续部分要根据项目本身的内部命名规范而变化。这些规范可指定某些目录名称的组成部分应该是公司、部门、项目、机器或逻辑名。

例如：

com.hundsun.intranet

com. hundsun.drms.tcm

1.2 变量

根据 Java Bean 规范，所有变量首字母小写，其余单词首字母大写，比如 projectManager myObject。对于一个 Bean 的成员变量，不允许出现 m_str 等前缀。

所以定义一个变量的代码如下：

Project myProject = new Project();

除临时变量和循环变量外或在变量的类型值不明确的情况下，要避免命名单个字符的局部变量或参数。习惯上，单个字符的命名为：

b for a byte

c for a char

d for a double

e for an Exception

f for a float

i, j, and k for integers

l for a long

o for an Object

s for a String

v for an arbitrary value of some type.

注：

1) 数组应该总是用下面的方式来命名：byte[] yBuffer;

2) 习惯上类成员变量的命名与一般变量命名不同，习惯上成员变量不加前缀，小写字母打头，用有意义的单词组成，多个单词时除第一个单词外首字母大写，如：projName,empName等

1.3 常量

如一般的编码规范所陈述的，常量以大写英文单词表示。

如：public static final int NUMBER = 2123;

1.4 类和接口

类和接口的名称应当为名词，采用大小写混用，内部每个单词首字母大写、其余全部小写的形式。尽量使你的类及接口的命名简单且具有描述性。要使用完整的单词——避免使用首字母缩写和缩写词 (除非缩写词的形式比它的完整形式使用更加普遍，例如URL或HTML)。

例如：

class Raster;

class ImageSprite;

1.5 方法

如一般的编程规范所陈述的，方法有如下命名结构： <Body>[Qualifier]。

方法名需为动词或动词短语，采用第一个字母小写、内部每个单词的首字母大写的大小写混用形式。

例如：

run();

runFast();

getBackground();

1.6 控件

我们推荐可视化控件的命名采用如下结构：

<Prefix><Body>[Qualifier]

下表定义了部分在 Jbuilder 中使用的 Java 控件的常用前缀。

Swing Page：

Control Item	Prefix
jButton	btn
jRadioButton	rb
jToggleButton	tb
jCheckBox	ckb
jLabel	lbl
jTextField	tf
jTextArea	ta
jTextPane	tp
jEditorPane	ep
jPasswordField	pf
jComboBox	cb
jList	l

jSlider	s
jProgressBar	pb
jScrollBar	sb
jTree	t
jTable	tbl
buttonGroup	bg
component	c

Swing container Page：

Control Item	Prefix
jPanel	pnl
jTabbedPane	tp
jScrollPane	scp
jSplitPane	spp
box	box
jMenuBar	mb
jPopupMenu	pm
jOptionPane	op
jColorChooser	cc
jFileChooser	fc

Data express Page：

Control Item	Prefix
database	db
tableDataSet	tds
textDataFile	tdf
queryDataSet	qds
queryResolver	qr
procedureDataSet	pd
procedureResolve	pr
parameterRow	prow
dataSetView	dsv
dataStore	ds
dataStoreConnection	dsc

txManager	txm
dataStoreServer	dss
dataStorePump	dsp
dataStoreSync	dssy

dbSwing Page：

Control Item	Prefix
jdbButton	dbtn
jdbRadioButton	dbrb
jdbToggleButton	dbtb
jdbCheckBox	dbckb
jdbLabel	dblbl
jdbTextField	dbtf
jdbTextArea	dbta
jdbTextPane	dbtp
jdbEditorPane	dbep
jdbPasswordField	dbpf
jdbComboBox	dbcb
jdbList	dbl
jdbSlider	dbs
jdbProgressBar	dbpb
jdbScrollBar	dbsb
jdbTree	dbt
jdbTable	dbtbl
tableScrollPane	tsp
jdbStatusLabel	dbsl
jdbNavToolBar	dbntb

2 编码规范

以下描述的是常用的编码结构：

2.1 变量的声明

要声明一个 Java 变量时，必须遵循以下格式：

1) 一个变量的声明用一行，因为这有利于注释。换句话说，

```
int level; // indentation level
int size; // size of table
```

要好于

```
int level, size;
```

2) 仅将声明放在代码块的开头。 (块就是由花括号“{” 和 “}”括起来的一些代码。)不要等到第一次使用它们时才来声明这些变量；这样会使粗心的程序员弄混，且在一定范围内阻碍代码的移植性。

```
void myMethod() {
    int int1 = 0; // 方法块的开头

    if (condition) {
        int int2 = 0; // "if" 块的开头
        ...
    }
}
```

3) 避免将声明隐藏在高层的局部声明。例如，不要在一个内部块中声明两个相同的变量名：

```
int count;
...
myMethod() {
    if (condition) {
        int count; // 避免！
```

```
            ...
        }
        ...
    }
```

2.2 类及接口的声明

编写Java类及接口时，应遵循以下格式规范：

1) 方法名与参数列前面的圆括号“(“ 之间不应有空格
2) 同一行的声明语句末尾应该有花括号“{”
3) 以花括号“}”开始的一行的缩进应与相应的开始语句对齐，以下情况除外：空语句中，花括号“{”后应紧接着有一个花括号“}”相对应。

```
class Sample extends Object {
    int ivar1;
    int ivar2;

    Sample(int i, int j) {
        ivar1 = i;
        ivar2 = j;
    }

    int emptyMethod() {}
    ...
}
```

4) 方法之间应以一行空行分隔开

2.3 复合语句

复合语句包含了一系列语句，这些语句以“{ 语句 }”的形式被封闭在花括号内。请参看以下部分的范例。

1) 被封闭的语句的缩进应该比混合语句多缩进一级。
2) “{” 应该放在复合语句第一行的末尾以标识混合语句的开始，而“}” 应该放在复合语句最后一行的开端，并且其缩进应与复合语句的开始位置对齐。
3) 所有的语句都应用花括号来封闭，即使是单一语句，当它们是控制结构的一部分时，也要用花括号来封闭，比如 if-else 语句或者 for 语句。这样，增加语句就更加容易，并且不会因忘记增加括号而产生意外的bug。

2.4 if, if-else-if else 语句

if-else 类型的语句应具有以下格式：

```
if (condition)
{
    statements;
}

if (condition)
{
    statements;
}
else
{
    statements;
}

if (condition)
{
```

```
        statements;
    }
    else if ( condition)
    {
        statements;
    }
    else
    {
        statements;
    }
```

注意: if 语句的程序体都要使用花括号 {}。应避免使用以下容易出错的格式：

```
if (condition) //AVOID! THIS OMITS THE BRACES {}!
    statement;
```

2.5 for 语句

for 语句应具有以下格式：

```
for ( initialization; condition; update)
{
    statements;
}
```

一条空的 for 语句(即所有的工作都在initialization子句、condition子句、update子句中完成)应该具有以下格式：

```
for ( initialization; condition; update);
```

当我们在 for 语句的initialization或者update子句中使用逗号操作符时，应避免使用三个以上的变量。如果情况需要如此，应在开始 for 循环之前（针对于initialization子句）或者在 for 循环结束后（针对于update子句）使用分隔语句。

2.6 while 语句

While 语句应具有以下格式:

```
while ( condition)
{
    statements;
}
```

一条空的 while 语句应具有以下格式：

```
while ( condition);
```

2.7 do-while 语句

do-while 语句应具有以下格式：

```
do {
    statements;
  } while ( condition);
```

2.8 switch 语句

switch 语句应具有以下格式：

```
switch ( condition) {
    case ABC:
        statements;
        /* falls through */
    case DEF:
        statements;
        break;
    case XYZ:
        statements;
        break;
    default:
```

```
            statements;
            break;
    }
```

每当一个 case 失败时（不包括 break 语句），在 break 语句通常所在的位置增加一行注释。如以上含有 /* falls through */ 的注释的代码范例所示。

每个 switch 语句都应包含一个default部分 。这个 部分中的 break 子句虽然有点多余，但它防止了以后增加另一个 case 时会发生失败错误。

2.9 try-catch 语句

try-catch 语句应具有以下格式：

```
try
{
        statements;
}
catch (ExceptionClass e)
{
        statements;
}
```

try-catch 语句后还可以跟 finally 子句， finally 子句不管 try 语句块是否全部成功执行，它都会执行。

```
try
{
    statements;
}
catch (ExceptionClass e)
{
    statements;
```

```
}
finally
{
    statements;
}
```

3 源文件编写规范

3.1 源文件命名

JAVA源程序是以.java为后缀，以能说明功能的英文单词命名，如果需要多个单词，则每个单词的第一个字母大写，如：EducationModify.java；个别单词过长可适当缩写，如：information可缩写成info，experience可以缩写成exp。

3.2 源文件的结构

文件头说明：见4.3.1例子

包含的文件：

如：package hotlava.net.stats;

import java.io.OutputStream;

import java.util.Observable;

import hotlava.util.Application;

package 行要在 import 行之前，import 中标准的包名要在本地的包名之前，而且按照字母顺序排列。

不允许出现类似import java.io.*这样的语句，开发人员可以借助开发环境的”Refactoring”功能自动组织import语句

常量定义： 定义在函数的开头部分。

类型定义：

全局变量定义：

类定义：

4 注释和 JavaDoc

注释是影响源代码的可读性和可维护性的关键因素。

Java 程序有两种类型的注释：执行注释和文件注释。执行注释就是象在 C++ 中，用/*…*/ 以及 // 符号分隔的注释。文件注释（即我们所说的 “doc comments”）只用于Java 程序，由 /**…*/ 来分隔。使用 Javadoc 工具可以将文件注释提炼成 HTML 文件。

以下描述的是常规注释：

4.1 包注释

每个包应该有一个命名为“package.html”的包注释文件，和JAVA源文件一起放在源文件树的同一个包目录下。应该包括对该包的简要描述。

4.2 类注释

一般的，一个源文件对应一个 Java 类，所以类注释也就是源文件的头注释。

类注释应写在每一个类声明的头部，其中包括的信息如下：

```
10  /**
11   * 该类负责项目相关对象的存取维护等操作
12   * <p>Copyright: Copyright (c) 2004</p>
13   * <p>Company: 杭州恒生电子有限公司</p>
14   * @author 钱晋
15   * @author $Author$
16   * @version 1.0, $Revision$
17   * @history
18   */
19  public abstract class ProjectManager {
```

注意：

(1) 要求开发人员把以上式样设置为开发环境的模版使用

(2) @history 不做强制要求，具体某段代码的 changeLog，通过 CVS 版本控制系统的 log 来体现。参见版本控制相关文档。

4.3 方法注释

方法注释包括方法功能描述，输入参数和返回参数及抛出异常的描述，如下图所示：

```
32  /**
33   * 创建一个新的项目
34   * 由于一个项目有很多信息，所以必须先填充好DTO对象，然后系统根据传入的DTO对象来创建Project
35   * @param dto <code>ProjectDTO</code>
36   * @throws NoPermissionException 如果不是管理员或相关授权人员，会抛出该异常
37   * @return Project 返回新创建的Project对象
38   */
39  public abstract Project createProject(ProjectDTO dto) throws
40      NoPermissionException;
```

如果 javadoc 的标记和真实方法的定义不一致，javaDoc 在自动生成文档的时候会出现错误，所以要求避免这种注释和代码不一致的情况。

4.4 行尾注释

行尾注释应使用于注释变量、常量的定义、条件语句、单一语句。行尾注释以 // 符号开始。

例如：

```
if (foo > 1) {
...
```

```
}
else{
    return false; // Explain why here.
}
```

4.5 嵌入式注释

嵌入式注释可以极大的提高程序的可读性，他们应当放在程序语句中的正确位置。如果程序的语句太长，将注释放在程序语句的顶部。除了循环变量之外，其他所有变量的声明都应该用嵌入式注释来说明。嵌入式注释应以“//”符号开始。

当过程或函数包含了大量的逻辑块时，将每一个逻辑块用单独的注释来说明，注释应与当前逻辑块的缩位对齐。尽量多使用以“//”开始的注释行，但不要以“*****”来分隔嵌入式注释。

一个嵌入式注释的范例：

例如：

```
if (foo > 1) {
// Do a double-flip.
...
}
else{
return false;
}
```

5 排版规范

5.1 大小写

1） Java 程序代码中所有的关键词都应当小写。例如：public, void.

2） 在 SQL 语句中所有的关键词都要大写。例如：SELECT, FOR, FROM, 等等。

5.2 代码缩进、空行及空格的使用

5.2.1 代码的缩进

代码缩进是提高源代码可读性和可维护性的重要手段。在 Java 编程中，2 个空格作为一个缩进单位，缩进一律使用 TAB 键。

5.2.2 行长度

要避免一行的长度超过80个字符，否则会使得许多终端和工具无法很好的对其处理。

一行长度最多不能超过100个字符

5.2.3 换行

当一个表达式无法在一行中完成时，要根据以下规则断开：

1）在逗号后断开。

2）在操作符前断开。

3）尽量选择较高级的断开而不是较低级别的断开。

4）将具有表达式开头的新行与前面的行在同一位置对准。

5）如果以上的规则使代码不清晰或导致代码挤压在右边，则可以缩进8格。

此处为一些断开的方法调用的例子：

```
someMethod(longExpression1,longExpression2,
longExpression3,
        longExpression4, longExpression5);
var = someMethod1(longExpression1,
        someMethod2(longExpression2,
                longExpression3));
```

以下是两个数学表达式换行断开的例子。我们更愿意选择第一种，因为断点设在了括号表达式的外面，这样级别更高。

```
longName1 = longName2 * (longName3 + longName4 -
```

```
longName5)
        + 4 * longname6; //  优先选择
longName1 = longName2 * (longName3 + longName4
        longName5) + 4 * name6; //  避免
```

以下是两个缩进方法声明的例子。第一种是常规的用法。第二种情况如果使用常规的缩进方法会第二和第三行代码挤压在右边，因此使用空8格可以使代码可读性更好。

```
//常规的缩进
someMethod(int anArg, Object anotherArg, String yetAnotherArg,
            Object andStillAnother) {
      ...
}
//缩进8格以避免缩进太深
private static synchronized horkingLongMethodName(int anArg,
          Object anotherArg, String yetAnotherArg,
          Object andStillAnother) {
  ...
}
```

if 语句的换行一般使用空8格的规则，因为常规的（4格）缩进会使主体部分不清晰。例如：

```
//不要使用这样的缩进
if ((condition1 && condition2)
    || (condition3 && condition4)
    ||!(condition5 && condition6)) { //不好的换行
    doSomethingAboutIt(); //MAKE THIS LINE EASY TO MISS
}
//而应使用这样的缩进
```

```
if ((condition1 && condition2)
        || (condition3 && condition4)
        ||!(condition5 && condition6)) {
    doSomethingAboutIt();
}
//或者使用这种缩进
if ((condition1 && condition2) || (condition3 && condition4)
        ||!(condition5 && condition6)) {
    doSomethingAboutIt();
}
```

此处是三种可接受的三重表达式的格式：

```
alpha = (aLongBooleanExpression) ? beta : gamma;
alpha = (aLongBooleanExpression) ? beta
                                 : gamma;
alpha = (aLongBooleanExpression)
        ? beta
        : gamma;
```

5.2.4 空行

空行可以通过将代码中逻辑相关的部分相分离来提高其可读性。

遇到以下情形时通常空两行：

1) 源文件的节与节之间

2) 类与接口的定义之间

遇到以下情形时通常空一行：

1) 方法与方法之间

2) 方法的局部变量与其第一个语句之间

3) 一个块或单行的注释之前

4) 方法中的不同逻辑部分间

5.2.5 空格

遇到以下情形时使用空格：

1) 与其后跟的括号之间要用一个空格分开。

例如:

```
while (true) {
    ...
}
```

但请注意，方法名与左括号之间不得使用空格。这有助于区分关键词与方法名。

2) 在参数列表中，逗号后要有空格。

3) 所有的二元操作符与操作数间要用空格分开。不能用空格来将一元的操作符与其操作数分开，如一元的减，自增(“++”), 和自减 (“--”)。

如:

```
a += c + d;
a = (a + b) / (c * d);
while (d++ = s++) {
    n++;
}
prints("size is " + foo + "\n");
```

4) 在for 语句的表达式中，要用空格分开。

如:

```
for (expr1; expr2; expr3)
```

附录 Web/Struts 编码规则

6.1 Web 目录命名规则

a. 不同的模块，按照其名称，分别建立独立的目录

b. 图片目录是/images

c. CSS 目录是/css

d. JavaScript 目录是/js

6.2 功能点 link 命名规则

a. Link 均以 动词+模块名称来命名。比如，"项目一浏"这个功能点的 link 是：listProject.do。又比如：进入创建项目页面的 link 是 addProject.do。

6.3 View/JSP 命名规则

由于使用了 tiles 框架，所以原则上 link 中不会再出现*.jsp 的联接。Jsp 文件只是纯粹的 view。所以 jsp 文件均要求放到 WEB-INF/pages 下。Jsp 文件的目录结构和 Web 目录结构一致。

对于 jsp 的名字命名规则，要和 link 的名字一致。比如 listProject.do 这个 link 对应的 jsp 就是 listProject.jsp.

对于 tiles definition 的命名规则，由于目前是一个 tile-definition 对应一个 jsp，所以 tiles definition 命名规则是：. + 目录名字 + . + … + . + JSP 页面的名字。

例 1：对于“项目一览”这个功能点：

用户输入的link是http://localhost:8080/project/listProject.do

定义一个 Action，Path 是/listProject，parameter 是”list”

对应的 tiles 定义是 .project.listProject

对应的 JSP 是/WEB-INF/pages/project/listProject.jsp

例 2：对于”修改某项目”这个功能点：

用户输入的link是http://localhost:8080/project/editProject.do?projectId=123

定义一个 Action，Path 是/editProject，在该 action 中进行初始化操作，完毕后 forward 到”.project.editProject”。

对应的 tiles 定义是.project.editProject

对应的 JSP 是/WEB-INF/pages/project/editProject.jsp

6.4 Action 命名规则

Action 按照功能可分为三种：

1) Forward Action

这些 Action 只是中转到一个 tile definition 或者 jsp 页面。这些 Action 通常是功能点的 link，命名参见 2.2。

2) Business Action

这些 Action 是调用业务层代码进行操作的地方。对于某个模块的操作，目前分类如下：

读操作(View Actions)：一览(list) 和 详细(detail)

写操作(Edit Actions)：创建(add), 保存(save) 和删除(delete)

比如，"项目管理这个功能点" 的 action 就可以定义 2 个类

ViewProjectAction.java, EditProjectAction.java

ViewProjectAction 中包括了 list 方法和 detail 方法。这 2 个方法分别映射成 listProject.do 和 detailProject.do 两个 action。(因为这两个 Action 分别是功能点的 link)。

EditProjectAction.java 包括了 addInit, add, saveInit, save, delete 5 个方法。，分别映射成 5 个 Action: addProjectInit.do, addProject.do, saveProjectInit.do, saveProject.do, deleteProject.do (这些 Action 基本上作为前一个 view 中 form 的 post action)

3) 中间操作的 action。

这些 Action 多用于准备数据，初始化环境等。一般的，这些 action 通过映射在 Business Action 中定义的某个方法来实现。(如果有必要单独创建一个 action class，需要和项目组长讨论确定)。这些 Action 的命名，是在对应的 Business Action 名字后添加 Init。比如：添加项目(addProject.do)前的准备操作，是 addProjectInit.do。即:

addProjectInit.do → .project.ProjectAdd → addProject.do

6.5 包命名规则

所有 Action 类放到 com.hs.drms.web.action 下，如果 Action 属于不同的模块，则需要新建立一个 package。

比如：

project 模块下的 AddProjectAction.java, 应该属于包 com.hs.drms.web.action.project。

类似的，project 模快下所有 Form 均放入 com.hs.drms.web.form.project 下.

对于一些常用的工具函数,可以放入 com.hs.drms.web.util.CommUtils.java 中. 这些工具函数均要求 public static。

6.6 Form 命名规则

一个 Action Form Bean Class 可以有多个 form 映射，一个 form 可以被多个 action 共享。所以这些情况下，Form Bean class 和 mapping 的命名有所区别：

ActionForm Bean 类名的命名规则：按照其职责/服务范围来命名。

例如：class FooForm{…}

ActionForm 和 Action 是 1:1 的关系：

即 一个 Form 只被一个 Action 使用， action form 的 Mapping 定义命名规则是：把 Mapping 看做是 ActionForm 的一个变量，即首字母小写的类名。

例如：ProjectForm.java 这个 Form 的 mapping 名称就是 projectForm

ActionForm 被多个 Action 共享：

action form 的映射命名规则是：Action 名称 + Form 的名称。

例如：

添加项目和修改项目共用一个 Action Form Class，这个类的名称是 ProjectForm.java。在 struts-config.xml 中有两个 form bean mapping。则映射的名称应分别为：addProjectForm 和 saveProjectForm。这样，在 validation.xml 中就可以针对 saveProjectForm 和 addProjectForm 分别执行不同的校验规则。

【修订记录】

批准日期	修订说明	编制部门
2007/09/12	初稿通过评审	EPG
2007/11/30	修改 logo	EPG